

LINEAMIENTO MAESTRO

Mapa conceptual integrador de Spring Boot

REST · Spring MVC · APIs · DTOs · Validation · Postman · PostgreSQL · JPA/Hibernate ·
Arquitectura Hexagonal · Relaciones 1:N · Errores · Transacciones

Guía paso a paso para estudiantes

Objetivo: consolidar todo lo necesario para ejecutar con seguridad los Capítulos 04 al 08 de ParkFlow 360.

PRODUCTO FINAL ESPERADO

Un mapa conceptual académico, técnico y explicativo - no un mapa mental decorativo - que muestre conceptos, relaciones, flujo de una petición, responsabilidades por capa, herramientas de trabajo y conexión directa con los Capítulos 04, 05, 06, 07 y 08.

Programación Aplicada 2026-2 · Proyecto ParkFlow 360

1. Propósito de la actividad

Esta actividad se realiza antes de continuar con la implementación de los Capítulos 04 al 08. El mapa conceptual servirá como un modelo de conocimiento del backend: cada estudiante debe ser capaz de mirar su mapa y explicar qué tecnología interviene, para qué sirve, cómo se conecta con las demás y en qué capítulo se aplica.

CRITERIO DOCENTE

El mapa debe permitir responder dos preguntas sin recurrir a apuntes: (1) “¿qué ocurre desde que un cliente HTTP envía una petición hasta que PostgreSQL guarda o consulta un dato?” y (2) “¿qué pieza de la arquitectura es responsable de cada decisión?”.

2. Resultado de aprendizaje

- Explicar la diferencia entre Internet, HTTP, REST, API, endpoint, JSON, cliente y servidor.
- Explicar el papel de Spring Boot, Spring Framework y Spring MVC en el backend.
- Distinguir Controller, DTO, caso de uso, dominio, Port IN, Port OUT, adapter de persistencia, entidad JPA y repository.
- Explicar la cadena JDBC -> JPA -> Hibernate -> Spring Data JPA -> PostgreSQL.
- Usar Postman o IntelliJ HTTP Client para construir y verificar peticiones REST.
- Comprender relaciones 1:N, PK, FK, @ManyToOne, @JoinColumn y FetchType.LAZY sin contaminar el dominio.
- Diferenciar validación de formato, regla de negocio e integridad de base de datos.
- Explicar manejo global de excepciones, códigos HTTP y @Transactional.
- Relacionar cada concepto con los Capítulos 04, 05, 06, 07 y 08.

3. Antes de dibujar: qué es un mapa conceptual

Un mapa conceptual organiza conocimiento mediante conceptos conectados por palabras de enlace. Cada conexión debe formar una proposición con sentido. Ejemplo: “Spring MVC recibe peticiones HTTP”, “Controller depende de Port IN”, “Hibernate implementa JPA” o “PostgreSQL garantiza integridad mediante constraints”.

NO ES UN MAPA MENTAL

No basta colocar palabras alrededor de un nodo central. Un mapa conceptual debe expresar relaciones semánticas. Cada flecha debe tener un verbo o frase de enlace: “usa”, “implementa”, “depende de”, “traduce”, “valida”, “persiste en”, “expone”, “protege”, “transforma”, “retorna”.

4. Metodología obligatoria de construcción

1. Inventario: listar primero todos los conceptos obligatorios de esta guía.
2. Jerarquización: ordenar desde conceptos generales hacia conceptos específicos.
3. Proposiciones: escribir las frases de enlace que explican cada relación.
4. Agrupación: dividir el mapa en bloques temáticos y visuales.

5. Cruces: conectar bloques que dependen unos de otros; por ejemplo, REST con Spring MVC, JPA con PostgreSQL y excepciones con códigos HTTP.
6. Trazabilidad: marcar cada concepto con C04, C05, C06, C07 o C08 según el capítulo donde se aplica.
7. Validación técnica: verificar que ninguna flecha implique una dependencia arquitectónica incorrecta.
8. Defensa oral: cualquier estudiante del grupo debe explicar el mapa de extremo a extremo.

5. Estructura maestra del mapa

Nodo central obligatorio: BACKEND PROFESIONAL PARKFLOW 360 CON SPRING BOOT Y ARQUITECTURA HEXAGONAL.

- Rama A - Fundamentos Web, HTTP, REST y APIs.
- Rama B - Spring Framework, Spring Boot, IoC/DI y Spring MVC.
- Rama C - Contrato HTTP: endpoints, DTOs, JSON, validación y códigos de estado.
- Rama D - Herramientas de prueba y desarrollo: Postman, IntelliJ HTTP Client, Maven, Git y DataGrip.
- Rama E - Persistencia: PostgreSQL, JDBC, ORM, JPA, Hibernate y Spring Data JPA.
- Rama F - Arquitectura hexagonal y patrones de diseño.
- Rama G - Modelado relacional y relación Cliente 1:N Vehículo.
- Rama H - Errores, validaciones de negocio y transacciones.
- Rama I - Flujo end-to-end de una petición ParkFlow 360.

REGLA VISUAL

Cada rama debe tener un color propio, pero el significado debe estar en las proposiciones, no en los colores. Se recomienda usar entre 80 y 140 nodos bien explicados antes que cientos de palabras sin relaciones claras.

PARTE I - Fundamentos Web, HTTP, REST y APIs

Esta parte prepara principalmente el Capítulo 04 y vuelve comprensible todo el flujo posterior.

Nodo / concepto obligatorio	Qué debe investigar y explicar	Relaciones que debe dibujar	Capítulo que habilita
Cliente / Servidor	Definir quién inicia la petición, quién procesa y quién responde. Diferenciar frontend, backend y base de datos.	Cliente -> envía petición a -> Servidor; Servidor -> devuelve -> respuesta.	C04-C08
HTTP / HTTPS	Qué es protocolo, request, response, método, URL, headers, body y status code. Diferencia conceptual HTTP/HTTPS.	HTTP -> transporta -> peticiones/respuestas; request -> contiene -> método, URL, headers, body.	C04-C08
REST	Qué significa estilo arquitectónico REST, recursos, URIs, representación, stateless y uso semántico de verbos HTTP.	REST -> modela -> recursos; recurso -> se identifica por -> URI; REST -> usa -> HTTP.	C04
API REST	Definir API, contrato, consumidor, proveedor, endpoint y versionado básico.	API -> expone -> capacidades; endpoint -> combina -> método + ruta.	C04-C08

Nodo / concepto obligatorio	Qué debe investigar y explicar	Relaciones que debe dibujar	Capítulo que habilita
Métodos HTTP	GET, POST, PUT, PATCH, DELETE: intención, idempotencia básica y cuándo no usarlos.	POST -> crea; GET -> consulta; PUT/PATCH -> modifica; DELETE -> elimina.	C04
Códigos HTTP	Familias 2xx/4xx/5xx. Dominar 200, 201, 204, 400, 404, 409, 500.	Resultado -> se comunica mediante -> status code.	C04,C08
JSON	Sintaxis objeto/array, clave-valor, tipos, serialización/deserialización.	JSON -> representa -> datos; Spring/Jackson -> transforma -> JSON <-> Java.	C04-C08

6.1 Investigación obligatoria de esta parte

- Buscar una definición académica de REST y explicar por qué REST no es “usar JSON” ni “hacer CRUD”.
- Investigar qué significa stateless y explicar un ejemplo sencillo.
- Investigar idempotencia de GET, PUT y DELETE y contrastarla con POST.
- Construir una tabla propia con al menos 10 códigos HTTP frecuentes y un ejemplo de ParkFlow 360.
- Dibujar una petición POST /api/clientes con headers, body JSON y respuesta 201.

PARTE II - Spring, Spring Boot, IoC/DI y Spring MVC

Nodo / concepto obligatorio	Qué debe investigar y explicar	Relaciones que debe dibujar	Capítulo que habilita
Spring Framework	Qué problema resuelve; contenedor IoC; beans; inyección de dependencias; estereotipos.	Spring -> administra -> beans; IoC -> invierte -> control de creación; DI -> entrega -> dependencias.	C04-C08
Spring Boot	Qué agrega sobre Spring: starters, auto-configuración, servidor embebido, configuración externa, arranque rápido.	Spring Boot -> simplifica -> configuración Spring; starter-web -> incluye -> MVC/Jackson/Tomcat.	C04-C08
IoC y DI	Constructor injection; por qué evitar new de dependencias de infraestructura en servicios.	ClienteService -> recibe -> ClienteRepositoryPort; Spring -> inyecta -> implementación.	C06-C08
Bean / Component Scan	@Component, @Service, @Repository, @RestController; escaneo desde paquete raíz.	Anotación -> registra -> bean; Boot -> escanea -> paquetes.	C04-C08
Spring MVC	Modelo request/response; DispatcherServlet de forma conceptual; controllers y mapping.	HTTP -> llega a -> Spring MVC -> despacha a -> Controller.	C04
Anotaciones MVC	@RestController, @RequestMapping, @GetMapping, @PostMapping, @PathVariable, @RequestParam, @RequestBody.	Anotación -> vincula -> parte de HTTP con método/argumento Java.	C04
ResponseBody	Controlar status, headers y body.	Controller -> devuelve -> ResponseEntity -> contiene -> status + body.	C04,C08

7.1 Investigación obligatoria

- Diferenciar Spring Framework de Spring Boot con un ejemplo.
- Explicar qué es un bean y quién controla su ciclo de vida.
- Explicar por qué constructor injection facilita pruebas y reduce acoplamiento.
- Investigar de forma conceptual qué papel cumple DispatcherServlet sin entrar todavía a su implementación interna.
- Relacionar cada anotación MVC con la parte exacta de una petición HTTP.

PARTE III - DTOs, contrato HTTP, validación y mapeo

Nodo / concepto obligatorio	Qué debe investigar y explicar	Relaciones que debe dibujar	Capítulo que habilita
DTO	Definir Data Transfer Object, Request DTO y Response DTO. Diferenciar DTO, dominio y entidad JPA.	Request JSON -> deserializa a -> Request DTO -> mapea a -> dominio.	C04,C06-C08
Java record	Qué es un record y por qué es adecuado para DTOs inmutables de transporte.	record -> reduce -> boilerplate DTO.	C04,C06-C08
Jakarta Validation	@Valid, @NotBlank, @NotNull, @Email, @Size, @Pattern; cuándo se ejecutan.	@Valid -> activa -> constraints; DTO -> protege -> frontera de entrada.	C04,C08
Validación de formato	Campos vacíos, formatos, tamaños, patrones.	DTO validation -> rechaza -> solicitud mal formada -> 400.	C04,C08
Validación de negocio	Duplicado, existencia de cliente, estado permitido.	Application Service -> valida -> reglas de negocio.	C06-C08
Mapper	Por qué traducir Web DTO <-> Domain y Domain <-> JPA Entity.	Mapper -> desacopla -> modelos de distintas fronteras.	C05-C08

CONEXIÓN CLAVE

El estudiante debe dibujar tres modelos diferentes: CrearClienteRequest / Cliente / ClienteJpaEntity. Debe escribir sobre las flechas “mapea hacia” y justificar por qué no son la misma clase.

PARTE IV - Herramientas que deben dominar

Nodo / concepto obligatorio	Qué debe investigar y explicar	Relaciones que debe dibujar	Capítulo que habilita
Postman	Workspace, collection, request, método, URL, params, headers, body raw JSON, Send, status, response body, variables de entorno.	Postman -> actúa como -> cliente HTTP -> prueba -> API.	C04-C08

Nodo / concepto obligatorio	Qué debe investigar y explicar	Relaciones que debe dibujar	Capítulo que habilita
IntelliJ HTTP Client	Archivos .http, separación ###, ejecución de requests reproducibles.	Archivo .http -> documenta/prueba -> endpoints.	C04-C08
Maven	pom.xml, dependency, starter, ciclo básico package/test, descarga de dependencias.	pom.xml -> declara -> starters/dependencies.	C04-C08
DataGrip / pgAdmin	Conexión, schema, consulta SQL, verificación de filas y constraints.	DataGrip -> inspecciona -> PostgreSQL.	C05-C08
Git	status, add, commit, push; evidencia por capítulo.	Cambio -> registra evidencia mediante -> commit.	C04-C08
IDE / Debugger	Breakpoints, variables, stack trace, logs.	Debugger -> permite observar -> flujo de ejecución.	C04-C08

9.1 Evidencia práctica obligatoria

- Captura o representación de un POST válido en Postman con 201.
- Representación de un POST inválido con 400.
- GET con PathVariable y GET con RequestParam.
- Consulta SQL en DataGrip que demuestre que un POST terminó en una fila real.
- Un ejemplo de archivo .http equivalente a una prueba de Postman.
- Explicación de para qué sirve cada herramienta dentro del ciclo de desarrollo.

PARTE V - PostgreSQL, JDBC, ORM, JPA, Hibernate y Spring Data JPA

Nodo / concepto obligatorio	Qué debe investigar y explicar	Relaciones que debe dibujar	Capítulo que habilita
PostgreSQL	Base, schema, tabla, fila, columna, PK, FK, UNIQUE, CHECK, NOT NULL, identity.	Tabla -> pertenece a -> schema; constraint -> protege -> integridad.	C05-C08
JDBC	API Java de bajo nivel para conexión y SQL. Driver PostgreSQL.	Spring/JPA -> utilizan indirectamente -> JDBC -> driver -> PostgreSQL.	C05
ORM	Qué problema intenta resolver y sus límites; objeto-relacional no elimina SQL.	ORM -> mapea -> objetos <-> tablas.	C05
JPA	Especificación; @Entity, @Id, @Table, @Column, @GeneratedValue, relaciones.	JPA -> define -> contrato de persistencia.	C05-C07
Hibernate	Implementación ORM de JPA; SQL generado; ddl-auto validate.	Hibernate -> implementa -> JPA; Hibernate -> genera/ejecuta -> SQL.	C05-C08
Spring Data JPA	JpaRepository, métodos derivados, save/findById/existsBy...	Spring Data -> abstraer -> repositorios JPA.	C05-C08

Nodo / concepto obligatorio	Qué debe investigar y explicar	Relaciones que debe dibujar	Capítulo que habilita
ddl-auto=validate	Por qué validar y no crear/actualizar una BD diseñada previamente.	Hibernate -> valida -> mapeo contra DDL existente.	C05
open-in-view=false	Concepto de límite del contexto de persistencia y por qué evitar fuga hacia web.	Configuración -> favorece -> límites de capa.	C05-C08

PROPOSICIÓN OBLIGATORIA

JPA es la especificación; Hibernate la implementa; Spring Data JPA simplifica repositorios; JDBC conecta finalmente con PostgreSQL. El mapa debe mostrar esta cadena completa y no tratarlos como sinónimos.

PARTE VI - Arquitectura hexagonal y patrones de diseño

Nodo / concepto obligatorio	Qué debe investigar y explicar	Relaciones que debe dibujar	Capítulo que habilita
Arquitectura Hexagonal	Objetivo: proteger lógica de negocio de detalles externos. Núcleo y adaptadores.	Dependencias -> apuntan hacia -> núcleo.	C06-C08
Port IN	Contrato de capacidades que el sistema ofrece; RegistrarClienteUseCase, ConsultarClienteUseCase.	Controller -> depende de -> Port IN.	C06-C08
Application Service / Use Case	Orquesta reglas y flujo del caso de uso.	Port IN -> implementado por -> Service; Service -> usa -> Port OUT.	C06-C08
Domain Model	Conceptos y reglas sin JPA/HTTP.	Dominio -> no depende de -> Spring MVC/JPA/PostgreSQL.	C06-C08
Port OUT	Contrato que el núcleo necesita del exterior.	Service -> depende de -> RepositoryPort.	C06-C08
Adapter IN	Controller web, DTOs, WebMapper.	HTTP -> entra por -> Adapter IN.	C06-C08
Adapter OUT	PersistenceAdapter, JPA Entity, Spring Data Repository.	Port OUT -> implementado por -> Adapter OUT.	C06-C08
Dependency Inversion	Módulo de alto nivel depende de abstracción, no del detalle técnico.	Use Case -> depende de -> interfaz; adapter -> implementa -> interfaz.	C06
Repository Pattern	Abstracción de acceso a agregados/datos; distinguir RepositoryPort de JpaRepository.	RepositoryPort -> abstraer -> persistencia; JpaRepository -> detalle técnico.	C06-C08
Adapter Pattern	Traduce una interfaz esperada a una tecnología concreta.	PersistenceAdapter -> adapta -> Spring Data a Port OUT.	C06-C08

Nodo / concepto obligatorio	Qué debe investigar y explicar	Relaciones que debe dibujar	Capítulo que habilita
-----------------------------	--------------------------------	-----------------------------	-----------------------

Mapper Pattern	Separa modelos por frontera.	Mapper -> transforma -> DTO/Domain/Entity.	C05-C08
----------------	------------------------------	--	---------

11.1 Regla de dependencia que debe aparecer literalmente

- ClienteController NO depende de SpringDataClienteRepository.
- ClienteService NO conoce ClienteJpaEntity.
- ClienteRepositoryPort NO extiende JpaRepository.
- Cliente de dominio NO tiene @Entity.
- ClientePersistenceAdapter Sí puede conocer Spring Data JPA porque está en infraestructura.

PARTE VII - Relación Cliente 1:N Vehículo

Nodo / concepto obligatorio	Qué debe investigar y explicar	Relaciones que debe dibujar	Capítulo que habilita
Cardinalidad 1:N	Un cliente puede tener muchos vehículos; cada vehículo pertenece a un cliente.	Cliente 1 -> se relaciona con -> N Vehículos.	C07
PK / FK	Dónde vive cliente_id; por qué FK está físicamente en vehiculo.	vehiculo.cliente_id -> referencia -> cliente.cliente_id.	C07
@ManyToOne	Anotación JPA en VehiculoJpaEntity; optional=false.	VehiculoJpaEntity -> muchos a uno -> ClienteJpaEntity.	C07
@JoinColumn	Mapea atributo Java a FK cliente_id.	@JoinColumn -> representa -> columna FK.	C07
FetchType.LAZY	Carga diferida y por qué no traer siempre todo el cliente.	LAZY -> posterga -> carga de asociación.	C07
Colaboración entre módulos	Vehicle consulta Customer mediante ConsultarClienteUseCase, no mediante repo interno.	VehiculoService -> usa -> ConsultarClienteUseCase.	C07
Integridad referencial	La aplicación valida antes, la FK sigue siendo última barrera.	Caso de uso -> valida -> existencia; PostgreSQL -> garantiza -> FK.	C07

TRES NIVELES DE LA MISMA RELACIÓN

Dominio: Vehiculo contiene clienteld. Persistencia JPA: VehiculoJpaEntity contiene @ManyToOne hacia ClienteJpaEntity. PostgreSQL: vehiculo.cliente_id es FOREIGN KEY. El mapa debe dibujar los tres niveles y explicar por qué no son equivalentes.

PARTE VIII - Errores profesionales, reglas de negocio y transacciones

Nodo / concepto obligatorio	Qué debe investigar y explicar	Relaciones que debe dibujar	Capítulo que habilita
Excepción de negocio	ClienteNoEncontradoException, PlacaDuplicadaException; no conocen HTTP.	Regla incumplida -> lanza -> excepción de negocio.	C06-C08
@RestControllerAdvice	Componente global que traduce excepciones a respuesta HTTP.	Exception -> manejada por -> GlobalExceptionHandler -> crea -> ApiError.	C08
@ExceptionHandler	Asocia tipo de excepción con tratamiento específico.	ExceptionHandler -> traduce -> tipo de fallo a status.	C08
ApiError	Contrato estable: timestamp, status, error, message, path, fieldErrors.	Global handler -> devuelve -> ApiError JSON.	C08
400 vs 404 vs 409 vs 500	Diferenciar validación de entrada, recurso inexistente, conflicto y fallo inesperado.	Tipo de error -> determina -> status HTTP.	C08
@Transactional	Atomicidad del caso de uso de escritura; commit/rollback conceptual.	Use case -> delimita -> transacción.	C08
Defensa en profundidad	Validación DTO + regla negocio + constraints PostgreSQL.	Aplicación -> valida antes; BD -> protege al final.	C08

PARTE IX - Flujo end-to-end obligatorio

El mapa debe contener un recorrido principal numerado. Utilice exactamente este orden conceptual y agregue las transformaciones que correspondan:

- Postman / Frontend envía HTTP POST /api/clientes con JSON.
- Spring Boot recibe la conexión; Spring MVC identifica el endpoint.
- ClienteController recibe la petición mediante @PostMapping y @RequestBody.
- Jackson deserializa JSON hacia CrearClienteRequest.
- @Valid ejecuta Jakarta Validation sobre el DTO.
- ClienteWebMapper transforma Request DTO hacia Cliente de dominio.
- Controller invoca RegistrarClienteUseCase (Port IN).
- ClienteService ejecuta reglas de negocio.
- ClienteService depende de ClienteRepositoryPort (Port OUT).
- ClientePersistenceAdapter implementa ese puerto.
- ClientePersistenceMapper transforma dominio hacia ClienteJpaEntity.
- SpringDataClienteRepository usa JPA/Hibernate.
- Hibernate genera/ejecuta SQL mediante JDBC contra PostgreSQL.
- PostgreSQL valida PK/UNIQUE/CHECK/FK y confirma la operación.
- El resultado vuelve: Entity -> Domain -> Response DTO -> JSON.
- Controller responde 201 Created.
- Si ocurre un fallo, la excepción llega al GlobalExceptionHandler y se transforma en ApiError + código HTTP adecuado.

DESAFÍO DE DEFENSA

El docente podrá señalar cualquier flecha del recorrido y preguntar: “¿qué entra?”, “¿qué sale?”, “¿quién conoce a quién?”, “¿qué patrón representa?” y “¿en qué capítulo se implementa?”.

PARTE X - Relación obligatoria con los Capítulos 04 al 08

Capítulo	Conceptos que deben aparecer conectados	Competencia consolidada
Capítulo 04 - Spring MVC real	HTTP, REST, API, endpoint, JSON, Spring MVC, Controller, Request/Response DTO, @PathVariable, @RequestParam, @RequestBody, @Valid, ResponseEntity, Postman/HTTP Client, 200/201/400/404.	El estudiante debe comprender el contrato HTTP antes de persistir.
Capítulo 05 - Primera persistencia real	PostgreSQL, schema, JDBC, ORM, JPA, Hibernate, Spring Data JPA, @Entity, @Table, @Column, @Id, @GeneratedValue, JpaRepository, mapper, ddl-auto=validate.	Debe entender cómo la petición termina en una fila real.
Capítulo 06 - Arquitectura hexagonal	Port IN, caso de uso, dominio, Port OUT, Persistence Adapter, inversión de dependencias, repository pattern, adapter pattern, mappers.	Debe poder dibujar la regla de dependencias.
Capítulo 07 - Relación 1:N	Cardinalidad, PK/FK, @ManyToOne, @JoinColumn, LAZY, colaboración Customer/Vehicle, reglas de existencia y duplicidad.	Debe diferenciar dominio, JPA y base de datos.
Capítulo 08 - Errores y transacciones	ApiError, @RestControllerAdvice, @ExceptionHandler, excepciones de negocio, 400/404/409/500, @Transactional, commit/rollback, validación por capas.	Debe convertir una API funcional en una API predecible y defendible.

PARTE XI - Protocolo de investigación

No se acepta un mapa construido solo copiando palabras de la presentación. Cada equipo debe investigar para poder explicar. Para cada concepto técnico deben completar internamente una ficha de investigación antes de dibujarlo.

26. Definición: explicar el concepto en palabras propias.
27. Problema que resuelve: qué dificultad existiría si no se utilizara.
28. Responsabilidad: qué hace y qué NO debe hacer.
29. Entrada y salida: qué recibe y qué produce.
30. Relaciones: con qué otros conceptos se conecta y mediante qué verbo.
31. Ejemplo ParkFlow 360: una clase, anotación, endpoint, tabla o flujo del proyecto.
32. Error frecuente: indicar al menos un uso incorrecto.
33. Capítulo: marcar C04, C05, C06, C07 o C08.
34. Fuente: registrar documentación oficial o material de clase consultado.

15.1 Fuentes sugeridas para investigar

- Documentación oficial de Spring y Spring Boot para Spring MVC, IoC/DI, Validation, Data JPA y transacciones.
- Especificación/documentación de Jakarta Persistence y Jakarta Validation para las anotaciones utilizadas.
- Documentación oficial de PostgreSQL para PK, FK, UNIQUE, CHECK, transacciones y schemas.
- Documentación oficial de Postman para construcción de requests y collections.
- RFC/MDN como referencia conceptual para métodos y códigos HTTP.
- Guías docentes de los Capítulos 04 al 08 como fuente del caso ParkFlow 360.

PARTE XII - Banco de palabras de enlace

Las flechas no deben quedar sin texto. Utilicen palabras de enlace precisas. Banco recomendado:

- “expone”, “consume”, “recibe”, “envía”, “serializa”, “deserializa”, “valida”, “mapea”, “traduce”, “implementa”, “depende de”, “inyecta”, “orquesta”, “persiste”, “consulta”, “protege”, “garantiza”, “referencia”, “adapta”, “lanza”, “captura”, “transforma”, “responde con”, “se ejecuta dentro de”, “se comunica mediante”.

EJEMPLOS DE PROPOSICIONES

Spring Boot -> configura y ejecuta -> aplicación Spring.
Spring MVC -> enruta -> request HTTP hacia Controller.
Controller -> depende de -> Port IN.
Service -> usa -> Port OUT.
PersistenceAdapter -> implementa -> Port OUT.
Hibernate -> implementa -> JPA.
VehiculoJpaEntity -> mapea FK mediante -> @ManyToOne + @JoinColumn.
GlobalExceptionHandler -> traduce -> excepción a ApiError + status HTTP.

PARTE XIII - Herramientas para construir el mapa

El equipo puede utilizar una herramienta digital que permita conectores etiquetados y exportación legible. Se recomiendan: CmapTools, diagrams.net (draw.io), Lucidchart, Miro o Canva. La herramienta es secundaria: lo evaluado es el conocimiento y la calidad de las relaciones.

- Formato final recomendado: PDF en tamaño A3 o superior, orientación horizontal.
- Debe poder leerse al 100 % sin textos diminutos.
- Usar colores por bloque temático y una leyenda C04-C08.
- No utilizar párrafos extensos dentro de los nodos; colocar definiciones de 1-3 líneas y relaciones claras.
- Usar íconos solo si ayudan a distinguir HTTP, código, base de datos y herramientas; no como decoración.

PARTE XIV - Paso a paso de ejecución para los estudiantes

35. Crear un lienzo horizontal grande y ubicar el nodo central.
36. Crear las nueve ramas maestras indicadas en la sección 5.
37. Agregar primero conceptos generales: HTTP, REST, API, Spring Boot, Spring MVC, PostgreSQL y arquitectura hexagonal.

38. Desplegar conceptos específicos debajo de cada uno: anotaciones, componentes, DTOs, puertos, adapters, constraints, etc.
39. Agregar verbos a TODAS las flechas principales.
40. Dibujar la cadena HTTP completa del Capítulo 04.
41. Dibujar la cadena de persistencia del Capítulo 05.
42. Dibujar el hexágono lógico y la dirección de dependencias del Capítulo 06.
43. Dibujar los tres niveles de Cliente 1:N Vehículo del Capítulo 07.
44. Dibujar el flujo de error y la frontera transaccional del Capítulo 08.
45. Agregar Postman, IntelliJ HTTP Client, DataGrip, Maven y Git en el punto donde realmente intervienen.
46. Agregar al menos 15 relaciones cruzadas entre ramas diferentes.
47. Revisar dependencias prohibidas y corregirlas.
48. Agregar códigos C04-C08 a cada bloque.
49. Preparar una defensa oral de 8-10 minutos donde el mapa se explique de izquierda a derecha y luego end-to-end.
50. Exportar a PDF y verificar legibilidad antes de entregar.

PARTE XV - Relaciones cruzadas obligatorias

- REST <-> HTTP: REST usa semántica HTTP para operar sobre recursos.
- JSON <-> DTO: JSON se transforma en Request DTO y Response DTO.
- Jakarta Validation <-> HTTP 400: fallas de formato se traducen a Bad Request.
- Spring MVC <-> Adapter IN: Controller es la entrada HTTP de la arquitectura.
- IoC/DI <-> arquitectura hexagonal: Spring conecta ports con adapters sin que el núcleo haga new del detalle.
- JPA/Hibernate <-> Adapter OUT: pertenecen a infraestructura, no al dominio.
- PostgreSQL constraints <-> reglas de negocio: ambos protegen, pero en niveles diferentes.
- @ManyToOne <-> FK: la anotación mapea una relación cuya integridad física está en PostgreSQL.
- Excepciones <-> GlobalExceptionHandler <-> códigos HTTP: semántica interna se traduce a contrato externo.
- @Transactional <-> Application Service: el caso de uso de escritura delimita la unidad atómica.
- Postman <-> API REST: Postman funciona como consumidor para probar el contrato.
- DataGrip <-> PostgreSQL: verifica la evidencia física de persistencia.
- Git <-> capítulos: cada avance queda trazable mediante commits.

PARTE XVI - Errores que invalidan el mapa

- Usar flechas sin palabras de enlace.
- Confundir REST con HTTP o con JSON.
- Presentar Spring Boot y Spring MVC como si fueran lo mismo.
- Presentar JPA, Hibernate y Spring Data JPA como sinónimos.
- Conectar Controller directamente con JpaRepository como diseño recomendado.
- Colocar @Entity en el modelo de dominio hexagonal.
- Decir que @ManyToOne crea la FK: la FK es una constraint física de PostgreSQL; JPA la mapea.
- Decir que @Valid ejecuta reglas como “placa duplicada”; esa es una regla de negocio.
- Decir que Postman es parte del backend; es una herramienta cliente de prueba.
- Dibujar @Transactional alrededor del Controller en vez del caso de uso.
- Usar 200 para todos los resultados y errores.
- Copiar definiciones sin relacionarlas con ParkFlow 360.

PARTE XVII - Entregables

- 1 PDF del mapa conceptual completo y legible.
- 1 archivo editable de la herramienta utilizada.
- 1 hoja de fuentes consultadas.
- 1 breve tabla de trazabilidad: concepto -> capítulo C04-C08.
- Defensa oral individual o grupal según instrucción docente.

PARTE XVIII - Rúbrica sugerida de evaluación

Criterio	Peso	Qué se espera
Cobertura conceptual	25%	Incluye todos los conceptos esenciales de HTTP/REST, Spring, MVC, persistencia, hexagonal, relaciones, errores y transacciones.
Calidad de relaciones	20%	Las flechas forman proposiciones correctas, precisas y explicables.
Comprensión arquitectónica	20%	Respetar dirección de dependencias, diferencia dominio/aplicación/infraestructura y ports/adapters.
Aplicación a capítulos 04-08	15%	Cada bloque está vinculado a ParkFlow 360 y a los capítulos correspondientes.
Investigación y precisión técnica	10%	Definiciones propias, fuentes fiables y ausencia de confusiones entre tecnologías.
Claridad visual y legibilidad	5%	Jerarquía clara, colores consistentes, textos legibles y navegación sencilla.
Defensa oral	5%	El estudiante explica recorrido end-to-end y responde preguntas sin depender del documento.

PARTE XIX - Checklist final antes de entregar

- ☐ El nodo central representa el backend ParkFlow 360.
- ☐ Están presentes HTTP, REST, API, endpoint, JSON y códigos HTTP.
- ☐ Están presentes Spring, Spring Boot, IoC/DI y Spring MVC.
- ☐ Están presentes DTOs, records, mappers y Jakarta Validation.
- ☐ Postman e IntelliJ HTTP Client aparecen como herramientas cliente.

- ☐ Están presentes PostgreSQL, JDBC, ORM, JPA, Hibernate y Spring Data JPA.
- ☐ Se entiende la cadena JPA -> Hibernate -> JDBC -> PostgreSQL.
- ☐ La arquitectura hexagonal muestra Adapter IN -> Port IN -> Use Case -> Port OUT -> Adapter OUT.
- ☐ No existen dependencias incorrectas hacia JpaRepository desde Controller o Service.
- ☐ La relación Cliente 1:N Vehículo aparece en dominio, JPA y BD.
- ☐ Se explican @ManyToOne, @JoinColumn y LAZY.
- ☐ Se explican validación de formato, negocio e integridad física.
- ☐ Se explican ApiError, ControllerAdvice, ExceptionHandler y 400/404/409/500.
- ☐ Se explica @Transactional y su ubicación en el caso de uso.
- ☐ El flujo end-to-end está numerado y completo.
- ☐ Todas las flechas principales tienen palabras de enlace.
- ☐ Hay al menos 15 relaciones cruzadas entre ramas.
- ☐ Cada bloque tiene trazabilidad C04-C08.
- ☐ El PDF final es legible y el estudiante puede defenderlo sin leer.

PARTE XX - Banco de preguntas para la defensa

1. ¿Cuál es la diferencia entre una API y un endpoint?
2. ¿Qué hace REST y qué hace HTTP?
3. ¿Qué diferencia existe entre @PathVariable y @RequestParam?
4. ¿Por qué Request DTO, dominio y JPA Entity no deberían ser la misma clase?
5. ¿Qué hace @Valid y qué no hace?
6. ¿Por qué POST de creación devuelve 201 y no siempre 200?
7. ¿Qué diferencia hay entre Spring Framework y Spring Boot?
8. ¿Qué problema resuelve IoC/DI?
9. ¿Cuál es la diferencia entre JPA, Hibernate y Spring Data JPA?
10. ¿Por qué ddl-auto=validate es coherente con una BD previamente diseñada?
11. ¿Qué diferencia existe entre Port IN y Controller?
12. ¿Qué diferencia existe entre Port OUT y JpaRepository?
13. ¿Qué patrón representa el PersistenceAdapter?
14. ¿Dónde vive físicamente cliente_id en la relación 1:N?
15. ¿Qué diferencia existe entre una FK y @ManyToOne?
16. ¿Por qué Vehicle consulta Customer mediante un caso de uso público?
17. ¿Qué diferencia existe entre 400, 404, 409 y 500?
18. ¿Por qué una excepción de negocio no debe conocer HTTP?
19. ¿Qué hace @RestControllerAdvice?
20. ¿Por qué @Transactional debe delimitar un caso de uso y no un Controller?
21. Explique todo el recorrido de POST /api/vehiculos desde Postman hasta PostgreSQL y de vuelta.

CONDICIÓN DE ÉXITO FINAL

El mapa está terminado cuando cualquier estudiante puede explicar los Capítulos 04 al 08 como un único sistema coherente: contrato HTTP, entrada Spring MVC, validación, caso de uso, dominio, ports/adapters, persistencia JPA/PostgreSQL, relaciones, errores y transacciones. El objetivo no es memorizar anotaciones, sino comprender responsabilidades, dependencias y flujo.